Analisis

Link al repositorio con los desafios:

https://github.com/Facundo-Mourelle/lab1-RTOS

Desafio 1

Al utilizar *xTaskCreate()*, creamos 4 tareas distintas que son delegadas al scheduler. Dado que este está configurado como **Preemptive** y cada tarea tiene la misma prioridad, se utiliza Round-Robin con intervalos de tiempo fijos. De esta forma, se le asigna a cada tarea una misma cantidad de tiempo de CPU. Sin embargo, como se utiliza HAL_Delay(), la tarea en ejecución consume todo su quantum realizando polling continuamente al systick timer, hasta que el scheduler realiza un cambio de contexto para pasar a la próxima tarea.

Así, las tareas permanecen alternando únicamente entre los estados *Running* y *Ready*, sin pasar nunca a *Blocked*, por lo que no se logra una concurrencia eficiente. Es por este motivo que los tiempos de conmutación de los LEDs se ven afectados en intervalos muy bajos, ya que el jitter aumenta debido a que el tiempo de espera depende de que la tarea recupere el control de la CPU para terminar su ciclo de polling.

```c
typedef struct {
  uint16_t leds;
  uint32_t delay_ms;
} blinkParams_t;

#define LED0 GPIO_PIN_0
#define LED1 GPIO_PIN_1
#define LED2 GPIO_PIN_2
#define LED3 GPIO_PIN_3
#define LED_PORT GPIOA

static blinkParams_t led1_params = {LED0, 200};
static blinkParams_t led2_params = {LED1, 400};
static blinkParams_t led3_params = {LED2, 600};
static blinkParams_t led4_params = {LED3, 800};

void vLedTask(void *pvParameters){
  blinkParams_t *params = (blinkParams_t *) pvParameters;
  for (;;) {
    HAL_GPIO_TogglePin(LED_PORT,params->leds);
    HAL_Delay(params->delay_ms);
  }
}

int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  xTaskCreate(vLedTask, "led1", 128, &led1_params, 0, NULL);
  xTaskCreate(vLedTask, "led2", 128, &led2_params, 0, NULL);
  xTaskCreate(vLedTask, "led3", 128, &led3_params, 0, NULL);
  xTaskCreate(vLedTask, "led4", 128, &led4_params, 0, NULL);
    while (1){}
}
```

Desafio 2

Transiciones de estado Al implementar *vTaskDelay()*, la tarea pasa a estado *Blocked* inmediatamente después de hacer toggle del led. Cuando las 4 tareas se encuentran bloqueadas, el scheduler detecta que no hay ninguna tarea en Ready y ejecuta Idle Task. Cuando el timer de tick llega al valor programado, el kernel mueve la tarea de la lista de bloqueados a la lista de Ready, y el scheduler la retoma. En este caso, las transiciones son: Running → Blocked → Ready → Running

En este escenario, la precisión mejora sustancialmente. El jitter ya no depende de la ejecución de un bucle de polling, porque la conmutación ocurre de forma determinista tras el desbloqueo de la tarea, permitiendo que el sistema aproveche los tiempos muertos de forma eficiente sin depender de que otras tareas consuman su quantum completo.

```c
typedef struct {
  GPIO_TypeDef *port;
  uint16_t pin;
  TickType_t delay;
} ledConfig_t;

#define LED0 GPIO_PIN_0
#define LED1 GPIO_PIN_1
#define LED2 GPIO_PIN_2
#define LED3 GPIO_PIN_3
#define LED_PORT GPIOA

static ledConfig_t cfg1 = {GPIOA, GPIO_PIN_0, pdMS_TO_TICKS(200)};
static ledConfig_t cfg2 = {GPIOA, GPIO_PIN_1, pdMS_TO_TICKS(400)};
static ledConfig_t cfg3 = {GPIOA, GPIO_PIN_2, pdMS_TO_TICKS(600)};
static ledConfig_t cfg4 = {GPIOA, GPIO_PIN_3, pdMS_TO_TICKS(800)};

void vTareaParpadeo(void *pvParameters) {
  const ledConfig_t *cfg = (const ledConfig_t *)pvParameters;
  for (;;) {
    HAL_GPIO_TogglePin(cfg->port, cfg->pin);
    vTaskDelay(cfg->delay);
  }
}

int main(void) {
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  xTaskCreate(vTareaParpadeo, "led1", 128,(void *)&cfg1 , 1, NULL);
  xTaskCreate(vTareaParpadeo, "led2", 128,(void *)&cfg2 , 1, NULL);
  xTaskCreate(vTareaParpadeo, "led3", 128,(void *)&cfg3 , 1, NULL);
  xTaskCreate(vTareaParpadeo, "led4", 128,(void *)&cfg4 , 1, NULL);
    while (1) {
    }
}
```

Desafio 3

Al configurar **POLL_MS = 0**, se produce un escenario de *Starvation*. Dado que la tarea del botón posee una prioridad mayor, al no invocar nunca un servicio de bloqueo como vTaskDelay(), el scheduler no encuentra motivos para desalojarla del estado Running. En consecuencia, la tarea del LED1 permanece eternamente en estado Ready sin acceder a la CPU, provocando que el LED quede estático y el sistema pierda su funcionalidad multitararea.

Al introducir un **POLL_MS** entre **1 y 10 ms**, se logra un equilibrio funcional. Cada llamada a vTaskDelay() transiciona la tarea del botón al estado *Blocked*, permitiendo que la tarea del LED1 entre en *Running* y complete sus ciclos de parpadeo. En términos de tiempo real, el tiempo de respuesta del LED2 queda determinado por el valor del delay: en el peor de los casos, la latencia será igual a POLL_MS (si el evento ocurre inmediatamente después de un sondeo). Por lo tanto, un valor de 10 ms es el límite máximo que satisface el deadline.

Si **POLL_MS** aumenta por encima de los **10ms**, el sistema recupera mayor disponibilidad de CPU para tareas bajas, pero se rompe el requisito de tiempo real. Con valores altos (ej. 100 ms), el parpadeo del LED1 es fluido, pero la respuesta del LED2 ante el pulsador se vuelve errática y notablemente lenta, ya que la latencia puede alcanzar el orden de magnitud del delay configurado, incumpliendo el objetivo de diseño.

```c
typedef struct {
  GPIO_TypeDef *port;
  uint16_t pin;
  TickType_t delay;
} ledConfig_t;

#define LED1 GPIO_PIN_13 // naranja
#define LED2 GPIO_PIN_15 // azul
#define LED_PORT GPIOD
#define BUTTON_PIN GPIO_PIN_0
#define BUTTON_PORT GPIOA
#define POLL_MS 10u

void vTareaLED(void *pvParameters) {

  (void)pvParameters;
  for (;;) {
    HAL_GPIO_TogglePin(LED_PORT, LED1);
    HAL_Delay(500);
  }
}

void vTareaBoton(void *pvParameters) {

  (void)pvParameters;
  for (;;) {
    if (HAL_GPIO_ReadPin(BUTTON_PORT, BUTTON_PIN) == GPIO_PIN_SET) {
      HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_SET);
    } else {
      HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_RESET);
    }
    vTaskDelay(pdMS_TO_TICKS(POLL_MS));
  }
}

int main(void) {
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  xTaskCreate(vTareaLED, "LED1", 128, NULL, 1, NULL);
  xTaskCreate(vTareaBoton, "BTN", 128, NULL, 2, NULL);
  while (1) {
  }
}
```

Desafio 4

Para cumplir con el período de 500ms, los argumentos de los servicios no deben ser iguales. En la *Tarea A* (vTaskDelay) el argumento debe ser 400ms, mientras que en la *Tarea B* (xTaskDelayUntil) debe ser 500ms. Esto es porque como cada función mide el paso del tiempo respecto a la ejecución del código.Si ambos argumentos fueran de 500ms, se observaría un time drift progresivo en el LED1 de la Tarea A. Mientras que xTaskDelayUntil toma como referencia el momento en que la tarea debió despertar, vTaskDelay simplemente añade el retardo al instante actual en que es llamada.Esto ocurre porque vTaskDelay no tiene en cuenta los 100ms consumidos por HAL_Delay. En consecuencia, el período real de la Tarea A sería la suma del tiempo de ejecución más el retardo solicitado (100ms + 500ms = 600ms), provocando que el LED1 parpadee más lento de lo requerido. Por el contrario, la Tarea B compensa automáticamente ese tiempo de ejecución, manteniendo el ciclo exacto de 500ms.

```c
#define LED1_PIN GPIO_PIN_12
#define LED2_PIN GPIO_PIN_13
#define LED_PORT GPIOD
#define PERIODO_MS 500u
#define CARGA_MS 100u

void vTareaA(void *pvParameters) {
  (void)pvParameters;
  for (;;) {
    HAL_GPIO_TogglePin(LED_PORT, LED1_PIN);

    HAL_Delay(CARGA_MS);

    /* 400ms de espera para intentar un período de 500ms */
    vTaskDelay(pdMS_TO_TICKS(PERIODO_MS - CARGA_MS));
  }
}

void vTareaB(void *pvParameters) {
  (void)pvParameters;
  TickType_t xLastWakeTime = xTaskGetTickCount();

  for (;;) {
    HAL_GPIO_TogglePin(LED_PORT, LED2_PIN);

    HAL_Delay(CARGA_MS);

    /* siempre espera hasta el próximo múltiplo de 500ms */
    vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(PERIODO_MS));
  }
}

int main(void) {
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  xTaskCreate(vTareaA, "LED1", 128, NULL, 1, NULL);
  xTaskCreate(vTareaB, "LED2", 128, NULL, 1, NULL);
  while (1) {
  }
}
```

Desafio 5

Al elevar la prioridad de la *Tarea A* por encima de la Tarea B, se produce un preemption. Durante los 3 segundos de prioridad superior, la *Tarea A* monopoliza el procesador cada vez que sale del estado **Blocked**, desplazando por completo a la Tarea B. Cuando la prioridad finalmente se restablece, el scheduler vuelve a aplicar Round-Robin y la *Tarea B* recupera el acceso a la CPU, retomando el parpadeo del LED2 desde el estado exacto en cuanto frenó. Para realizar el conteo de los 3 segundos se utilizo el servicio *vTaskDelay()*. Es fundamental que este conteo se realice mediante una función bloqueante, ya que si se intentara usar un bucle de espera activa (como HAL_Delay), la Tarea A al tener mayor prioridad se quedaría en estado **Running** acaparando la CPU y no permitiría la ejecución de ninguna otra tarea de menor prioridad durante ese intervalo.

No se logro en el primer intento el comportamiento del enunciado ya que ambas tareas utilizan el mismo tiempo de bloqueo (300ms). La *Tarea A* solo logra congelar a la *Tarea B* si esta intenta entrar en **Running** mientras la *Tarea A* todavía tiene prioridad alta y requiere CPU. En caso contrario, si ambas están en **Blocked** el efecto de "congelado" es menos evidente.

```c
#define LED1_PIN   GPIO_PIN_12   /* Tarea A */
#define LED2_PIN   GPIO_PIN_13   /* Tarea B */
#define LED_PORT   GPIOD
#define BTN_PIN    GPIO_PIN_0
#define BTN_PORT   GPIOA

#define PERIODO_MS     300u
#define BOOST_MS      3000u
#define PRIORIDAD_BASE   1u

static TaskHandle_t xHandleTareaA = NULL;

void vTareaA(void *pvParameters) {
  (void)pvParameters;
  for (;;) {
    HAL_GPIO_TogglePin(LED_PORT, LED1_PIN);
    if (HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN) == GPIO_PIN_SET) {
      UBaseType_t prio = uxTaskPriorityGet(NULL);
      /* Aumento de prioridad*/
      /*queda por debajo → se congela */
      vTaskPrioritySet(NULL, prio + 1);
      /* mantiene prioridad elevada durante 3 segundos*/
      vTaskDelay(pdMS_TO_TICKS(BOOST_MS));
      /* restaura prioridad original*/
      vTaskPrioritySet(NULL, PRIORIDAD_BASE);
    }
    vTaskDelay(pdMS_TO_TICKS(PERIODO_MS));
  }
}

void vTareaB(void *pvParameters) {
  (void)pvParameters;
  for (;;) {
    HAL_GPIO_TogglePin(LED_PORT, LED2_PIN);
    vTaskDelay(pdMS_TO_TICKS(PERIODO_MS));
  }
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    xTaskCreate(vTareaA, "LED1", 128, NULL, PRIORIDAD_BASE, &xHandleTareaA);
    xTaskCreate(vTareaB, "LED2", 128, NULL, PRIORIDAD_BASE, NULL);
    vTaskStartScheduler();
    while(1){
    }
}
```

Desafio 6

Para visualizar que esta tarea no consume tiempo del procesador mientras espera, se puede observar la actividad de la **Idle Task** mediante un LED. Dado que *vTareaLED* permanece en estado **Blocked** gran parte del tiempo, el scheduler no tiene tareas en **Ready** y el procesador se mantiene ejecutando la tarea de idle. Esto se ve reflejado en el LED que permanece encendido durante todo este tiempo.

El único momento en que la Idle Task cede la CPU es cuando la interrupción despierta a vTareaLED. En ese instante, el procesador sale del estado de reposo para realizar la conmutación del LED, y una vez que la tarea vuelve a ejecutar *xSemaphoreTake*, el sistema regresa inmediatamente a la tarea idle. Como este proceso ocurre en una fracción de tiempo, el LED apenas muestra parpadeo, confirmando que el uso de recursos por parte de la aplicación es prácticamente nulo mientras no hay eventos, por lo que no se desperdician practicamente ciclos del cpu.

```c
#define LED3_PIN GPIO_PIN_14
#define LED_PORT GPIOD
#define BTN_PIN GPIO_PIN_0
#define BTN_PORT GPIOA

static SemaphoreHandle_t xSemBinario = NULL;

void vTareaLED(void *pvParameters) {
  (void)pvParameters;
  for (;;) {
    /* Bloquea indefinidamente hasta recibir el semáforo */
    xSemaphoreTake(xSemBinario, portMAX_DELAY);
    /* ISR hizo Give → conmuta LED */
    HAL_GPIO_TogglePin(LED_PORT, LED3_PIN);
  }
}

void IRQHandler(void) {
  BaseType_t xHigherPrioTask = pdFALSE;

  xSemaphoreGiveFromISR(xSemBinario, &xHigherPrioTask);
  /* Si vTareaLED tiene mayor prio que la tarea interrumpida → context switch
   * inmediato*/
  portYIELD_FROM_ISR(xHigherPrioTask);

  /* Limpia el flag de la línea EXTI*/
  __HAL_GPIO_EXTI_CLEAR_IT(BTN_PIN);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    xSemBinario = xSemaphoreCreateBinary();
    xTaskCreate(vTareaLED, "LED3", 128, NULL, 1, NULL);
    vTaskStartScheduler();
    while (1){
    }
}
```

Desafio 7

Para alterar el orden de la secuencia, se debe modificar la lógica de desboqueo entre las tareas mediante la reasignación de los semáforos. Cada tarea está bloqueada esperando un semáforo específico y, al terminar su acción, libera el semáforo que desbloquea a la siguiente. Es perfectamente posible implementar una solución escribiendo una única función de tarea, siempre que se utilicen correctamente los parámetros de entrada. Se puede definir una estructura que contenga el semáforo que la tarea debe esperar, el semáforo que debe liberar y el pin del LED que debe controlar. Al crear las tres instancias de la tarea con xTaskCreate(), se le pasa a cada una un puntero a su estructura correspondiente. Esto permite reutilizar el mismo código.

```c
#define LED1_PIN GPIO_PIN_12
#define LED2_PIN GPIO_PIN_13
#define LED3_PIN GPIO_PIN_14
#define LED_PORT GPIOD

#define DELAY_MS 500u
static SemaphoreHandle_t xSem12 = NULL;
static SemaphoreHandle_t xSem23 = NULL;
static SemaphoreHandle_t xSem31 = NULL;

void vTarea1(void *pvParameters) {
  (void)pvParameters;
  for (;;) {
    xSemaphoreTake(xSem12, portMAX_DELAY);

    HAL_GPIO_WritePin(LED_PORT, LED1_PIN, GPIO_PIN_SET);
    vTaskDelay(pdMS_TO_TICKS(DELAY_MS));
    HAL_GPIO_WritePin(LED_PORT, LED1_PIN, GPIO_PIN_RESET);

    xSemaphoreGive(xSem23);
  }
}

void vTarea2(void *pvParameters) {
  (void)pvParameters;
  for (;;) {
    xSemaphoreTake(xSem23, portMAX_DELAY);

    HAL_GPIO_WritePin(LED_PORT, LED2_PIN, GPIO_PIN_SET);
    vTaskDelay(pdMS_TO_TICKS(DELAY_MS));
    HAL_GPIO_WritePin(LED_PORT, LED2_PIN, GPIO_PIN_RESET);

    xSemaphoreGive(xSem31);
  }
}

void vTarea3(void *pvParameters) {
  (void)pvParameters;
  for (;;) {
    xSemaphoreTake(xSem31, portMAX_DELAY);

    HAL_GPIO_WritePin(LED_PORT, LED3_PIN, GPIO_PIN_SET);
    vTaskDelay(pdMS_TO_TICKS(DELAY_MS));
    HAL_GPIO_WritePin(LED_PORT, LED3_PIN, GPIO_PIN_RESET);

    xSemaphoreGive(xSem12);
  }
}

int main(void) {
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  xSem12 = xSemaphoreCreateBinary();
  xSem23 = xSemaphoreCreateBinary();
  xSem31 = xSemaphoreCreateBinary();

  xSemaphoreGive(xSem12);

  xTaskCreate(vTarea1, "T1", 128, NULL, 1, NULL);
  xTaskCreate(vTarea2, "T2", 128, NULL, 1, NULL);
  xTaskCreate(vTarea3, "T3", 128, NULL, 1, NULL);
  vTaskStartScheduler();
  while (1) {
  }
}
```

Desafio 8

Para contar los tiempos dentro de la secuencia protegida, se deben utilizar servicios bloqueantes, ya que aunque el Mutex garantiza la exclusividad sobre los LEDs, el uso de *vTaskDelay()* permite que el procesador ejecute otras tareas de menor prioridad o la **Idle Task** durante los intervalos entre encendidos. Si se utilizara *HAL_Delay()*, la tarea que posee el Mutex acapararía el 100% de la CPU en un bucle de espera activa, impidiendo que cualquier otra tarea progrese, a pesar de que el procesador no está realizando ningún trabajo útil.

Si se reescribe el ejercicio utilizando *HAL_Delay()*, el comportamiento visual de la secuencia se mantendría idéntico, pero el sistema dejaria de ser multitarea. Al no haber bloqueos reales, la tarea que no posee el Mutex permanecería en estado Ready consumiendo tiempo de CPU cada vez que el scheduler le da un quantum, revisa el Mutex cerrado y vuelve a esperar. El sistema se volvería ineficiente y la latencia aumentaría drásticamente para todas las demas tareas secundarias.

```c
#define LED1_PIN  GPIO_PIN_12
#define LED2_PIN  GPIO_PIN_13
#define LED3_PIN  GPIO_PIN_14
#define LED4_PIN  GPIO_PIN_15
#define LED_PORT  GPIOD

#define STEP_MS   150u

/* Cambiar tipo de delay */
#define DELAY(ms)   HAL_Delay(ms)
/* #define DELAY(ms)   vTaskDelay(pdMS_TO_TICKS(ms)) */
static SemaphoreHandle_t xMutexLEDs = NULL;

void vTareaA(void *pvParameters)
{
    (void)pvParameters;
    const uint16_t leds[] = { LED1_PIN, LED2_PIN, LED3_PIN, LED4_PIN };
    for (;;)
    {
        xSemaphoreTake(xMutexLEDs, portMAX_DELAY);
        /*seccion critica */
        for (int i = 0; i < 4; i++)
        {
            HAL_GPIO_WritePin(LED_PORT,
                              LED1_PIN|LED2_PIN|LED3_PIN|LED4_PIN,
                              GPIO_PIN_RESET);
            HAL_GPIO_WritePin(LED_PORT, leds[i], GPIO_PIN_SET);
            DELAY(STEP_MS);
        }
        HAL_GPIO_WritePin(LED_PORT,
                          LED1_PIN|LED2_PIN|LED3_PIN|LED4_PIN,
                          GPIO_PIN_RESET);

        xSemaphoreGive(xMutexLEDs);

        /*fuera de seccion critica*/
        DELAY(STEP_MS);
    }
}

void vTareaB(void *pvParameters)
{
    (void)pvParameters;
    for (;;)
    {
        xSemaphoreTake(xMutexLEDs, portMAX_DELAY);
        /* seccion critica */
        for (int i = 0; i < 4; i++)
        {
            HAL_GPIO_WritePin(LED_PORT,
                              LED1_PIN|LED2_PIN|LED3_PIN|LED4_PIN,
                              GPIO_PIN_SET);
            DELAY(STEP_MS);
            HAL_GPIO_WritePin(LED_PORT,
                              LED1_PIN|LED2_PIN|LED3_PIN|LED4_PIN,
                              GPIO_PIN_RESET);
            DELAY(STEP_MS);
        }

        xSemaphoreGive(xMutexLEDs);

        DELAY(STEP_MS);
    }
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    xMutexLEDs = xSemaphoreCreateMutex();
    xTaskCreate(vTareaA, "TA", 128, NULL, 1, NULL);
    xTaskCreate(vTareaB, "TB", 128, NULL, 1, NULL);

    vTaskStartScheduler();
    while (1){
    }
}
```

Desafio 9

La elección entre polling e interrupción para determinar el estado del pulsador impacta directamente en la eficiencia energética y el determinismo del sistema. Con polling, la *Tarea B* debe ejecutarse periódicamente para consultar el estado del pin, lo que consume ciclos de CPU. Con interrupciones el sistema ignora por completo el evento hasta que ocurra el cambio de flanco físico, liberando al procesador y eliminando la latencia.

La solución que genera el mejor tiempo de respuesta es la basada en interrupciones, ya que al presionar el boton, el hardware suspende la tarea actual de forma inmediata para ejecutar la ISR. Sin embargo, con polling el tiempo de respuesta esta limitado por el intervalo del *vTaskDelay()* de la tarea de monitoreo.

```c
#define LED1_PIN GPIO_PIN_12
#define LED2_PIN GPIO_PIN_13
#define LED3_PIN GPIO_PIN_14
#define LED4_PIN GPIO_PIN_15
#define LED_PORT GPIOD
#define BTN_PIN GPIO_PIN_0
#define BTN_PORT GPIOA

#define STEP_MS 200u

static TaskHandle_t xHandleTareaA = NULL;

void vTareaA(void *pvParameters) {
  (void)pvParameters;
  const uint16_t leds[] = {LED1_PIN, LED2_PIN, LED3_PIN, LED4_PIN};
  uint8_t i = 0;
  for (;;) {
    HAL_GPIO_WritePin(LED_PORT, LED1_PIN | LED2_PIN | LED3_PIN | LED4_PIN,
                      GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, leds[i], GPIO_PIN_SET);
    i = (i + 1) % 4;
    vTaskDelay(pdMS_TO_TICKS(STEP_MS));
  }
}

void vTareaB(void *pvParameters) {
  (void)pvParameters;
  uint8_t estadoAnterior = GPIO_PIN_RESET;
  uint8_t suspendida = 0;

  for (;;) {
    uint8_t estadoActual = HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN);
    if (estadoActual == GPIO_PIN_SET && estadoAnterior == GPIO_PIN_RESET) {
      if (!suspendida) {
        vTaskSuspend(xHandleTareaA);
        suspendida = 1;
      } else {
        vTaskResume(xHandleTareaA);
        suspendida = 0;
      }
    }
    estadoAnterior = estadoActual;
    vTaskDelay(pdMS_TO_TICKS(20));
  }
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    xTaskCreate(vTareaA, "LEDS", 128, NULL, 1, &xHandleTareaA);
    xTaskCreate(vTareaB, "BTN",  128, NULL, 2, NULL);

    vTaskStartScheduler();
    while (1) {
    }
}
```

Desafio 10

Una primera indicacion de que la tarea se ha eliminado es revisando si LED4 efectivamente se encuentra encendido, ya que es la tarea inmediatamente anterior a ejecutarse antes de la eliminacion de la tarea *vTareaLightShow()*. Otra forma válida de corroborar esto sería como en desafios anteriores, mediante un LED que se encienda a partir de **Idle Task**, puesto que es la tarea por defecto a ejecutar cuando no hay tareas en **READY**.

```c
#define LED1_PIN  GPIO_PIN_12
#define LED2_PIN  GPIO_PIN_13
#define LED3_PIN  GPIO_PIN_14
#define LED4_PIN  GPIO_PIN_15
#define LED_PORT  GPIOD

#define DURACION_SHOW_MS  10000u
#define STEP_MS           100u

void vTareaLightShow(void *pvParameters) {
  (void)pvParameters;
  const uint16_t patron[] = {0,
                             LED1_PIN | LED3_PIN,
                             LED2_PIN | LED4_PIN,
                             LED1_PIN | LED3_PIN,
                             LED2_PIN | LED4_PIN,
                             0,
                             LED1_PIN,
                             LED1_PIN | LED2_PIN,
                             LED2_PIN | LED3_PIN,
                             LED3_PIN | LED4_PIN,
                             LED4_PIN | LED3_PIN,
                             LED3_PIN | LED2_PIN,
                             LED2_PIN | LED1_PIN,
                             0};
  const uint8_t PASOS = sizeof(patron) / sizeof(patron[0]);

  TickType_t xInicio = xTaskGetTickCount();
  uint8_t paso = 0;

  /* animacion durante 15 segundos */
  while (xTaskGetTickCount() - xInicio < pdMS_TO_TICKS(DURACION_SHOW_MS)) {
    HAL_GPIO_WritePin(LED_PORT, LED1_PIN | LED2_PIN | LED3_PIN | LED4_PIN,
                      GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, patron[paso], GPIO_PIN_SET);
    paso = (paso + 1) % PASOS;
    vTaskDelay(pdMS_TO_TICKS(STEP_MS));
  }

  /* apaga todo y deja LED4 encendido*/
  HAL_GPIO_WritePin(LED_PORT, LED1_PIN | LED2_PIN | LED3_PIN | LED4_PIN,
                    GPIO_PIN_RESET);
  HAL_GPIO_WritePin(LED_PORT, LED4_PIN, GPIO_PIN_SET);

  /* bon voyage */
  vTaskDelete(NULL);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    xTaskCreate(vTareaLightShow, "SHOW", 256, NULL, 1, NULL);

    vTaskStartScheduler();
    while (1) {
    }
}
```